이음 E : UM

배치 실행, 서버 관제, 업무 관제를 하나로 잇는 사내 운영 도구

지금은 배치를 돌리는 스케줄러, 서버를 보는 관제, 업무 처리를 보는 관제가 따로 있습니다. 이음은 이 셋을 실행 파일 하나로 다시 만들고, 실행마다 **실행 번호**를 붙여 모든 기록을 한 줄로 잇습니다.

이음이라는 이름은 흩어진 도구를 하나로 잇고, 배치 실행과 그 뒤의 기록을 잇는다는 뜻입니다.

사용자: 배치·서비스 운영 담당자, 개발자

대상: 폐쇄망 리눅스 서버

결과물: 하루 만에 만드는 개념 검증판

Q1 그래서 대체 뭘 만드는 건가요?

배치 스케줄러, 서버 관제, 업무 관제에 함수 단위 추적을 더한 운영 도구입니다. 실행 파일 하나로 설치합니다.

2~3쪽

Q2 기존의 뭘 보완하는 건가요?

따로 쓰는 도구 3개 사이의 빈틈입니다. 서로 이어지지 않아 확인이 번거롭고, 데이터는 제품 안에 갇혀 있으며, 비용은 세 갈래로 나갑니다.

4~6쪽

Q3 뭐가 좋아지는 건가요?

운영 도구가 하나로 줄고, 배치 흐름을 업무 상황과 함께 보며, 지나간 장애도 원인 함수까지 확인하고, 쌓인 데이터를 분석에 씁니다.

7~13쪽

Q4 어떻게 가능한 건가요?

스케줄러가 실행마다 번호를 붙여 프로세스에 넘기면 모든 기록이 그 번호로 이어집니다. 어려운 부분은 리눅스 perf, SQLite 같은 검증된 부품에 맡기고, 2명이 AI 세션 4개로 하루에 검증판을 만듭니다.

14~15쪽

Q1 그래서 대체 뭘 만드는 건가요?

도구 3개가 나눠 하던 일을 실행 파일 하나로 다시 만듭니다.

배치 스케줄러, 서버 관제, 업무 관제를 합치고, 지금은 맡는 도구가 없는 함수 단위 추적을 더합니다. 해커톤 결과물은 이 넷이 실제로 이어져 도는 개념 검증판입니다.

배치 스케줄러
정해진 시각에 작업 실행
지금: 상용 Control-M

서버 관제
CPU·메모리·프로세스
지금: 상용 onTune

업무 관제
주문 접수·처리·응답시간
지금: 사내 개발 APM

함수 단위 추적
CPU를 쓴 코드 기록
지금: 없음 — 새로 추가

→

이음

- 실행: 플로우·선후행·재시도·원격 실행 — 번호
- 자원: 서버와 실행별 CPU·메모리, 1초 — 번호
- 업무: 접수·처리·대기·응답시간, 1초 — 번호
- 함수: CPU를 쓴 함수, 10초 구간 — 번호

한 DB, 공개 스키마 · 실행 이력 · 서버 자원 · 실행별 자원 · 업무 지표 · 함수 기록

웹 화면
대시보드·플로우·타임머신

SQL 조회
읽기 전용

내보내기
CSV·함수 기록

설치 모양

매니저 1대
스케줄러, 저장, 웹 화면

서버마다 에이전트
실행, 자원·업무 지표 수집, 함수 기록

같은 실행 파일
역할만 다르게 실행

운영 요구	이음의 방식
폐쇄망	외부 서비스와 CDN 없이 동작. 화면 자원까지 실행 파일에 포함
방화벽	에이전트가 매니저로만 연결. 매니저 포트 1개만 열면 됨
실행 통제	검토된 작업 정의(YAML)에 있는 명령만 실행. 에이전트는 토큰으로 인증
서버 부담	함수 추적은 지정한 작업에만 켬. 추적 자체의 CPU도 화면에 표시

한 줄 정의

작업을 띄우는 일과 그 작업을 지켜보는 일을 한 도구가 맡습니다.

Q1 그래서 대체 뭘 만드는 건가요? 하루 운영에 얹어 보기

하루 운영 흐름에 얹어 보면 이렇게 씁니다.

증권 주문 체결 업무 기준입니다. 새벽 배치가 체결 서비스를 띄우고, 그 서비스가 장중 내내 주문을 처리하며, 장 마감 후 정산 배치가 돕니다.

	05:30 개장 준비	09:00~15:30 장중	15:40 정산	언제든 돌아보기
실행 스케줄러	**개장 준비 플로우 실행** 기준정보 적재 2개를 동시에 돌리고, 둘 다 끝나면 체결 서버에서 체결 서비스를 띄움. 실패하면 자동 재시도	**서비스는 계속 실행 중** 띄운 스크립트는 끝났지만 서비스는 같은 번호로 계속 추적	**정산 플로우 실행** 체결 대기가 0건이 된 것을 확인한 뒤 시작 확장 목표	실행 이력과 재시도 기록이 남음
관제 서버·업무	적재 배치의 CPU·메모리, 서버 자원	**서버 자원 + 업무 지표 1초** 주문 접수·처리 건수, 처리 대기, 응답시간을 서버 자원과 한 화면에	정산 배치의 자원, 체결 대기 건수	지난 구간의 지표를 다시 펼쳐 봄
기록 같은 번호	**번호 발급** 01JBQ7… 체결 서비스에 번호가 넘어감	**서비스의 모든 기록에 같은 번호** 자원, 업무 지표, CPU를 쓴 함수(10초 구간)	정산 실행도 각자의 번호로 기록	보존 기간은 우리가 정함
활용 판단·분석	어느 단계가 늦었는지 플로우 그래프로 확인	**체결이 밀리면** 응답시간 타일에서 그 서비스의 기록으로 바로 이동	정산 종료 시각 추이 확인	**SQL·CSV로 분석** 용량 판단, 배치 종료 시각 예측, 배포 전후 비교

같은 **실행 번호**가 새벽 배치에서 장중 서비스로, 장 마감 후 분석까지 이어집니다. 그래서 스케줄러와 관제가 한 도구여야 합니다.

Q2 기존의 뭘 보완하는 건가요?

도구마다 제 몫은 하지만, 도구 사이가 비어 있습니다.

배치 실행, 서버 자원, 업무 처리 현황을 각각 다른 도구가 봅니다. 셋을 잇는 공통 키가 없고, 어느 코드가 CPU를 썼는지는 맡는 도구가 없습니다.

배치 스케줄러

상용 · Control-M

무엇을, 언제 실행했고 성공했는가

단위: 작업

자체 화면 · 자체 저장소

서버 관제

상용 · onTune

어느 서버, 어느 프로세스가 CPU·메모리를 썼는가

단위: 서버·프로세스

자체 화면 · 자체 저장소

업무 관제

사내 개발 APM

주문이 몇 건 들어와 처리됐고, 응답이 얼마나 걸렸는가

단위: 업무 프로세스

자체 화면 · 자체 저장소

공통 키 없음 → 사람이 시각과 서버 이름으로 맞춤

어느 **함수(코드)**가 CPU를 썼는가 → 맡는 도구 없음. 필요할 때 개발자가 서버에 접속해 직접 측정

데이터는 **제품마다 따로** 쌓이고, 형식·보존 기간·꺼낼 수 있는 범위를 제품이 정함

도구별로 비어 있는 곳과 이음이 채우는 것

도구	비어 있는 곳	이음이 채우는 것
배치 스케줄러	띄운 서비스가 그 뒤 어떻게 도는지 모름. 업무가 바쁜지와 상관없이 시각이 되면 실행	실행 번호를 서비스까지 이어 추적. 업무 지표를 실행 조건으로 확장
서버 관제	그 프로세스를 어느 실행이 띄웠는지, CPU를 어느 함수가 썼는지 모름	실행 번호로 자원을 실행에 연결. 함수 단위 상시 기록
업무 관제	처리가 밀린 원인이 업무량인지, 자원인지, 코드인지 모름	업무 지표, 자원, 함수 기록을 같은 시간축에
세 도구 공통	데이터가 제품 안에 있고, 계약·서버·화면이 세 벌	한 DB와 공개 스키마, 실행 파일 1개

보완하는 것

도구 하나하나가 아니라, 도구 사이의 연결과 데이터입니다.

Q2 기존의 뭘 보완하는 건가요? 빈틈이 만드는 불편

그 빈틈 때문에 운영에서 세 가지가 불편합니다.

불편 1

특이상황을 확인하기 번거롭다

장중 주문이 몰려 체결이 밀린 경우, 지금은 이렇게 따라갑니다.

1	업무 관제	응답시간 상승, 처리 대기 증가 확인
2	운영 담당	그 서비스가 어느 서버, 어느 프로세스인지 확인
3	서버 관제	그 시각 CPU 확인. "프로세스가 CPU를 다 썼다"에서 멈춤
4	스케줄러	그 서비스를 띄운 실행, 같은 시각의 다른 작업 확인
5	개발자	몇 분 만에 지나간 폭주라 **같은 부하를 다시 만들어 재현**한 뒤 측정

도구 3개를 오가는 5단계, 원인 코드는 재현부터

불편 2

쌓인 데이터를 활용하기 어렵다

운영에서 정말 궁금한 질문은 두 도구 이상의 데이터를 엮어야 답할 수 있습니다.

- 초당 몇 건부터 체결이 밀리나? 서버를 늘릴 때인가?
  업무 지표 + 자원
- 밀린 구간에 CPU를 쓴 함수는 평소와 무엇이 달랐나?
  업무 지표 + 함수 기록(지금은 없음)
- 정산 배치가 끝나는 시각이 매달 늦어지고 있나?
  실행 이력 + 자원
- 배포 뒤 같은 작업의 함수별 CPU 비중이 달라졌나?
  실행 이력 + 함수 기록

통계·머신러닝은 엮인 원자료가 있어야 시작

불편 3

비용이 많이 든다

매일 쓰는 기능은 정해진 시각 실행, 선후행, 재시도, 원격 실행, 로그, 서버·업무 지표 정도입니다. 비용은 제품 단위로 나갑니다.

스케줄러 라이선스·유지보수	상용 · 매년
서버 관제 라이선스·유지보수	상용 · 매년
업무 관제(사내 APM) 유지보수	사내 인력
제품별 관리 서버·저장소	3벌
계약 갱신·교육·문의 창구	제품마다

제품 밖의 기능이 필요하면 벤더 개발이나 별도 연동을 기다려야 합니다.

쓰는 기능은 일부, 비용은 세 갈래

Q2 기존의 뭘 보완하는 건가요? 보완 방법

연동으로는 빈틈이 메워지지 않아, 하나로 새로 만듭니다.

보완하는 방법을 네 가지로 놓고 비교했습니다. 기준은 기록을 실행 번호로 이을 수 있는지, 데이터를 우리가 쥘 수 있는지입니다.

방법	실행 번호로 잇기	함수 단위 기록	데이터를 SQL로 직접	비용	폐쇄망·기능 추가
A. 지금 도구 + 연동 개발	△ 제품끼리 번호를 주고받지 못해 시각으로 맞춤	×	△ 제품별 형식	높음 기존 + 연동비	△ 벤더 일정
B. 오픈소스 조합 스케줄러+관제+프로파일러	△ 셋을 잇는 코드를 결국 직접 작성	△ 별도 도구	○ 저장소 3개	낮음	△ 제품 3개 운영
C. 대형 통합 관측 제품 상용 APM·관측 플랫폼	△ 서비스·서버는 잇지만 배치 스케줄러가 없음	△ 제품에 따라	△ 벤더 저장소	매우 높음	△ 폐쇄망 제약
D. 하나로 새로 (이음)	○ 띄우는 쪽이 번호를 심음	○ 같은 번호·같은 DB	○ 공개 스키마 1개	낮음 서버 +담당자 시간	○ 코드가 사내에

B를 택해도 세 제품을 잇는 데이터 모델과 연동 코드는 직접 만들어야 합니다. 이 과제의 핵심이 그 부분이므로 처음부터 하나로 만드는 편이 단순합니다.

번호는 띄우는 순간에만 붙일 수 있습니다

택배 송장번호처럼, 물건을 보낼 때 붙여야 이후 기록이 그 번호로 조회됩니다. 배치가 서비스를 띄울 때 번호를 넘겨야 그 서비스의 자원, 업무 지표, 함수 기록이 같은 번호를 답니다.

자체 구현의 부담도 계산에 넣습니다

유지보수는 우리 몫입니다. 그래서 비용 비교에 담당자 인건비를 넣고, 검증된 범위부터 병행 운영으로 적용합니다(13쪽).

그래서

띄우는 쪽(스케줄러)과 재는 쪽(관제)이 한 제품이어야 번호가 끊기지 않습니다.

Q3 뭐가 좋아지는 건가요?

좋아지는 것은 네 가지입니다.

장애 대응만 달라지지 않습니다. 평소 운영, 배치 관리, 데이터 활용까지 함께 달라집니다. 각 항목은 다음 쪽의 예상 화면으로 보여 드립니다.

① 운영이 단순해집니다

지금 도구 3개, 상용 계약 2건, 관리 서버 3벌, 화면 3개

이음 **실행 파일 1개, 화면 1개, 방화벽 포트 1개**

화면 ① 대시보드 · 8쪽

② 배치를 한눈에 보고, 업무에 맞춰 돌립니다

지금 작업 목록과 로그 위주. 업무가 바쁜지와 상관없이 시각이 되면 실행

이음 **갈래·합류·서버 넘기·재시도를 그래프로. "체결 대기 0건 뒤 정산" 같은 조건 실행**

확장

화면 ② 플로우 그래프 · 9쪽

③ 지나간 장애도 원인 함수까지 봅니다

지금 도구 3개를 오가는 5단계. 원인 코드는 재현한 뒤에야 확인

이음 **화면 1개에서 2단계. 이미 지나간 구간도 재현 없이 함수까지**

화면 ③ 타임머신 · ④ 플레임 그래프 · 10~11쪽

④ 쌓인 데이터가 자산이 됩니다

지금 제품이 보여 주는 화면과 보고서 범위 안에서만 확인

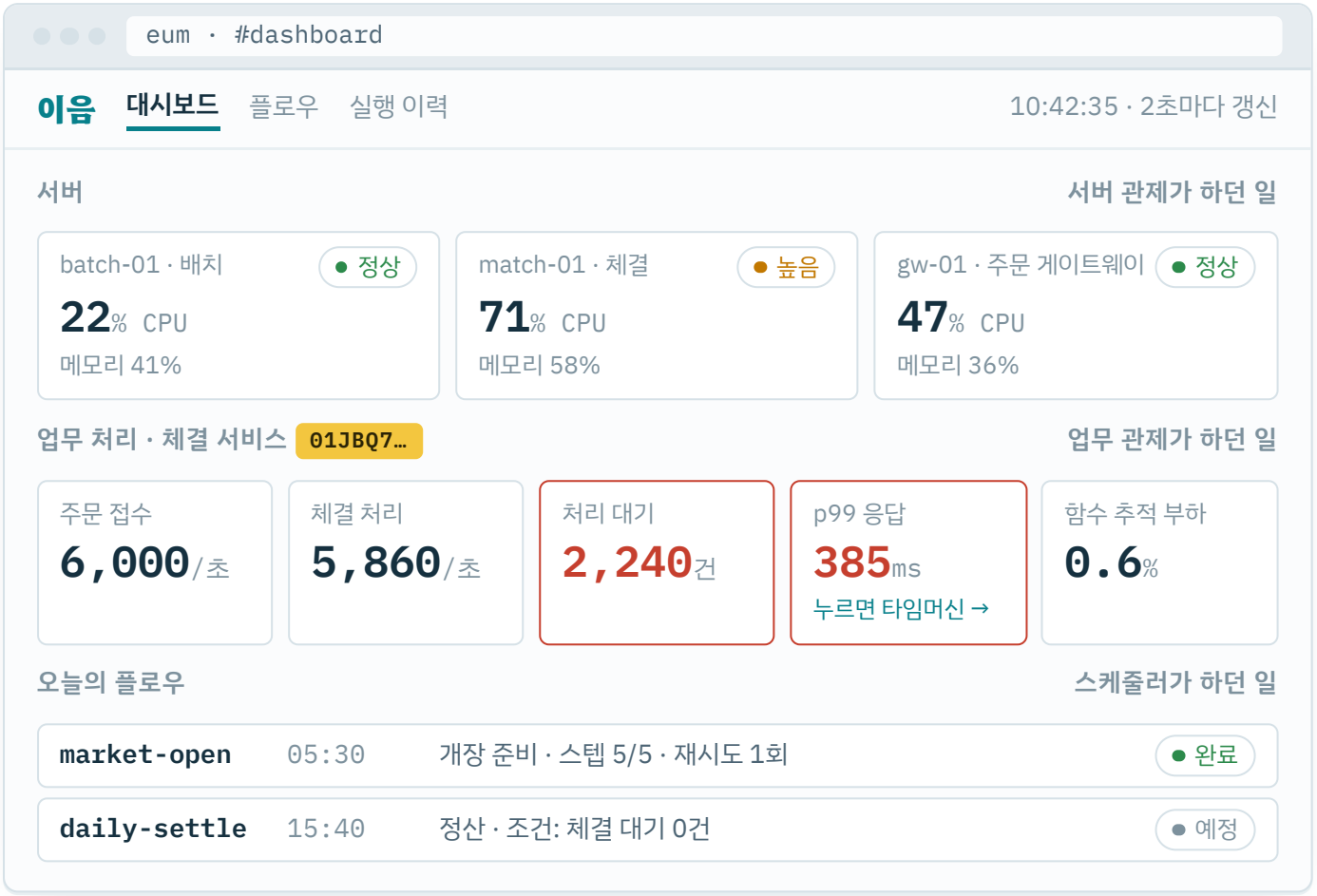
이음 **운영 데이터 5종을 SQL 한 번으로 조인. CSV로 꺼내 통계·머신러닝에**

화면 ⑤ 데이터 꺼내기 · 12쪽

Q3 뭐가 좋아지는 건가요? ① 운영이 단순해집니다

한 화면에서 서버, 업무, 배치를 함께 봅니다.

지금은 세 도구의 화면을 따로 열어야 볼 수 있는 내용입니다.



설계 시안 · 수치는 합성 부하 기준 예시

무엇을 보나

서버 자원, 주문 접수·처리·대기·응답시간, 오늘의 배치 흐름을 한 화면에서 봅니다.

어떻게 쓰나

업무 지표에는 그 서비스를 띄운 실행 번호가 붙어 있습니다. 응답시간 타일을 누르면 그 실행의 기록으로 바로 갑니다.

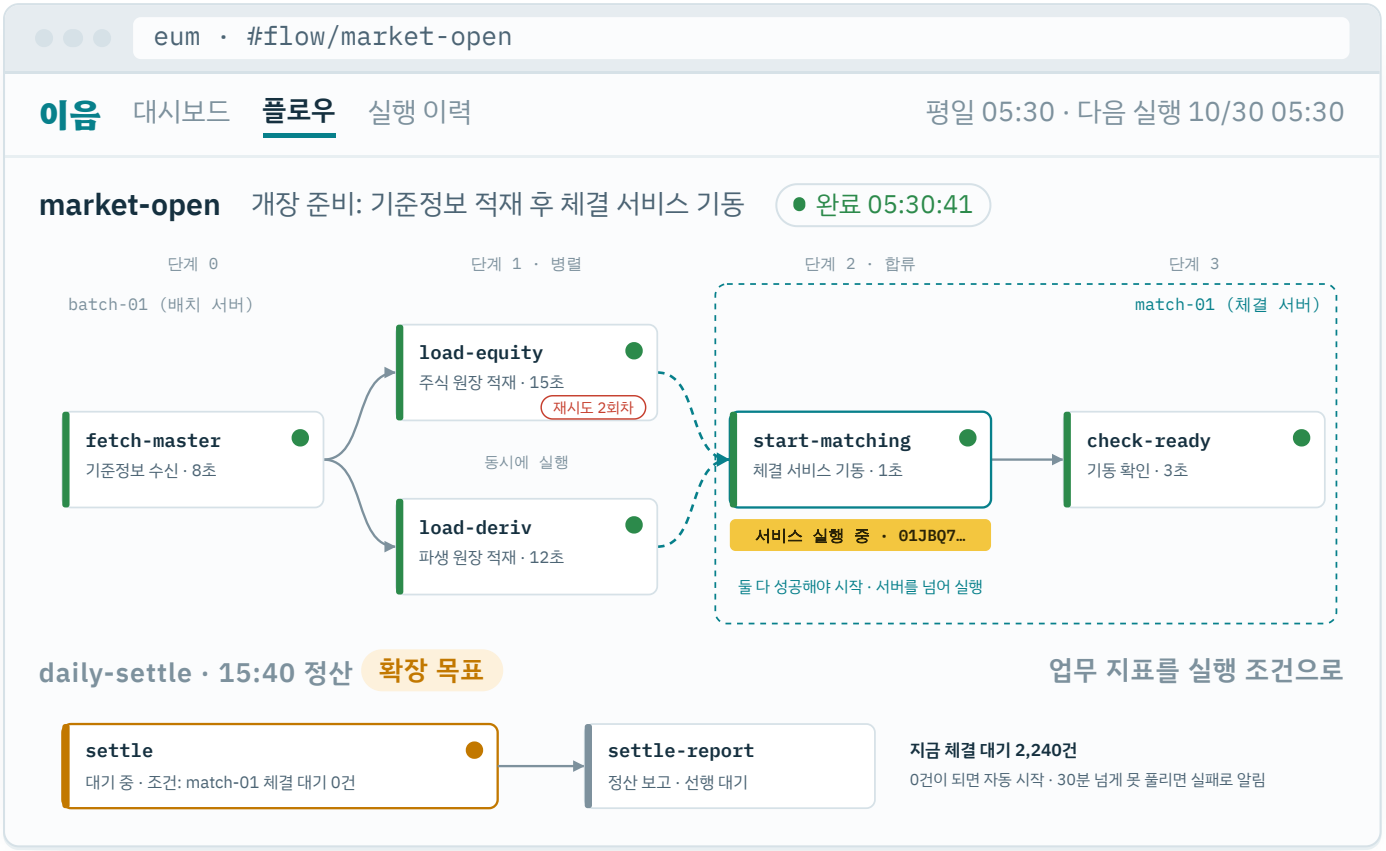
좋아지는 점

화면 3개를 오가던 일상 점검이 화면 하나로 줄어듭니다.

Q3 뭐가 좋아지는 건가요? ② 배치를 한눈에, 업무에 맞춰

배치 흐름을 그래프로 보고, 업무 상황에 맞춰 실행합니다.

작업이 갈라지고(병렬), 합쳐지고(합류), 서버를 넘는 흐름과 재시도를 한눈에 봅니다.



설계 시안 · 노드를 누르면 그 실행의 기록(타임머신)으로 이동

무엇을 보나

선후행, 병렬과 합류, 서버를 넘는 실행, 재시도 이력. 서비스를 띄운 스텝은 끝나도 서비스가 살아 있는 동안 같은 번호로 계속 기록합니다.

업무를 아는 스케줄러

업무 지표가 같은 DB에 있으므로 "체결 대기가 0건이 된 뒤 정산 시작" 같은 조건을 추가 개발 없이 겁니다. 지표를 모르면 실행하지 않는 쪽으로 판단합니다.

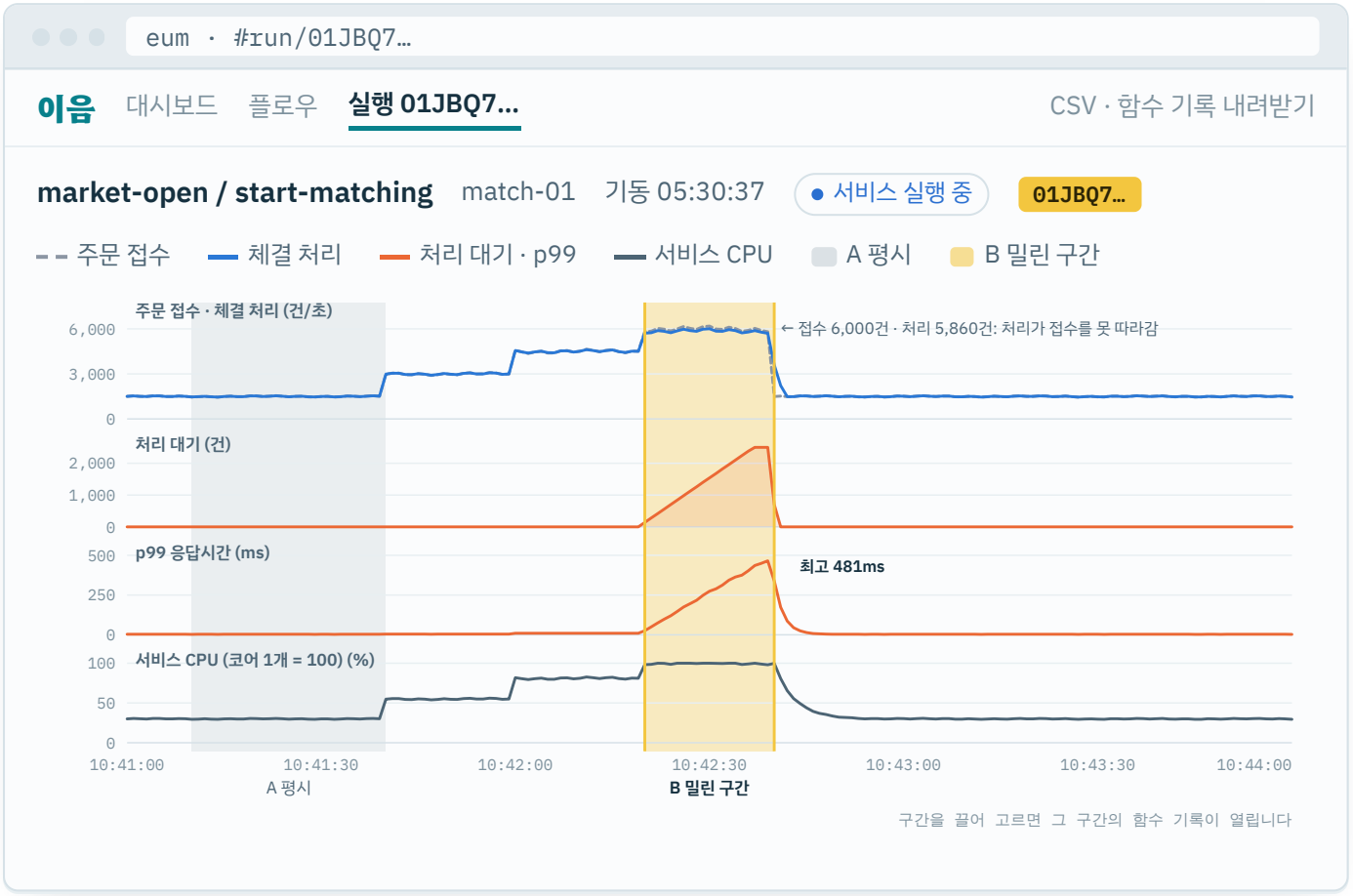
좋아지는 점

배치가 시각만 보지 않고 업무 상황을 보고 돕니다.

Q3 뭐가 좋아지는 건가요? ③ 지나간 장애도 원인 함수까지

느려졌던 그 순간을, 지나간 뒤에도 다시 펼쳐 봅니다.

장중 주문이 1,500건/초에서 6,000건/초로 몰린 3분을 그 서비스의 실행 번호로 모아 한 시간축에 놓았습니다.



설계 시안 · 시연용 C 체결 서비스에 합성 주문으로 만든 부하 기준 예시

무엇을 보나

그 실행이 띄운 서비스의 업무 지표, 서비스 CPU, 서버 자원을 같은 시간축에 겹칩니다. 도구마다 시각을 맞출 필요가 없습니다.

어떻게 쓰나

차트에서 평시 구간과 밀린 구간을 끌어 고르면, 그 구간에 CPU를 쓴 함수가 다음 쪽 플레임 그래프로 나옵니다.

좋아지는 점

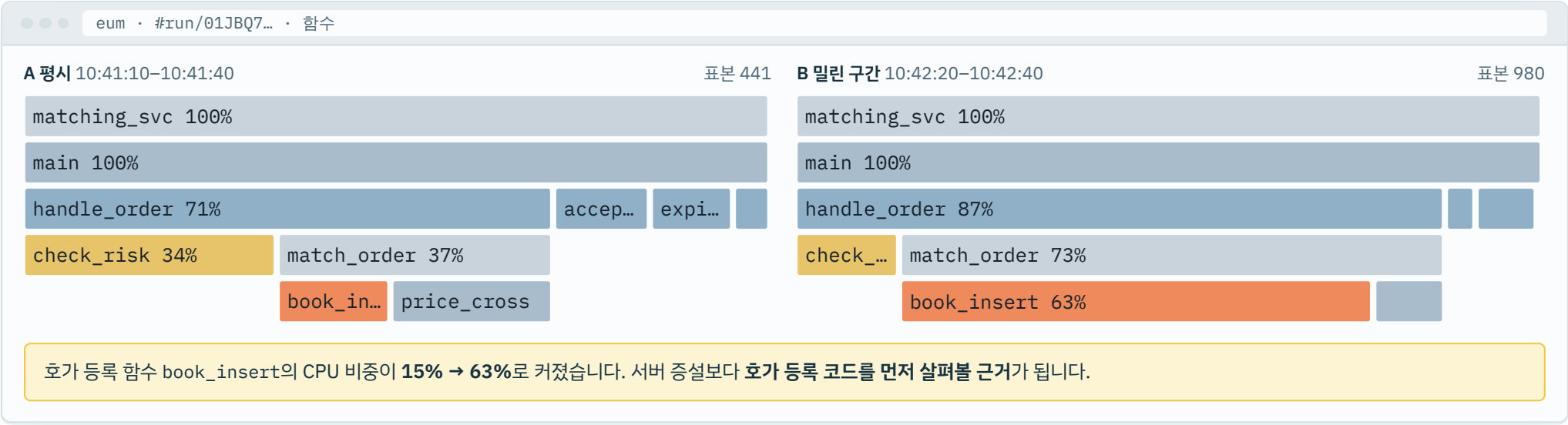
5단계 → 2단계. 타일 클릭, 구간 선택이면 함수까지 갑니다.

단계 수는 절차 설계 기준입니다. 걸리는 시간은 같은 시나리오로 실측해 비교합니다(13쪽).

Q3 뭐가 좋아지는 건가요? ③ 지나간 장애도 원인 함수까지

그 구간에 CPU를 쓴 함수까지 내려가, 재현 없이 근거를 얻습니다.

위가 바깥 함수, 아래로 갈수록 안쪽 함수입니다. 막대 폭이 CPU를 쓴 비율입니다.



호가 등록 함수 book_insert의 CPU 비중이 **15% → 63%**로 커졌습니다. 서버 증설보다 **호가 등록 코드를 먼저 살펴볼 근거**가 됩니다.

장애가 다르면, 함께 보는 기록도 다릅니다

상황	이음에서 함께 보는 기록	먼저 답할 질문
배치가 실패함	종료 코드, 로그 끝부분, 선행 작업 결과, 재시도 이력	어느 단계에서 실패했고, 앞 작업과 관련이 있나?
서비스가 내려감	실행 번호로 추적하던 프로세스 수, 직전 자원 변화	언제 사라졌고, 직전에 무슨 일이 있었나?
응답이 느려짐	처리량·대기·응답시간, 서버·서비스 자원, 함수별 CPU	업무량, 자원, 코드 중 어디부터 볼까?

범위를 분명히 합니다

함수 기록은 CPU를 쓴 곳만 보여 줍니다. 잠금이나 디스크를 기다려 느린 경우는 이번 범위 밖이고, CPU 비중만으로 원인을 확정하지 않습니다.

좋아지는 점

개발자가 재현 환경을 만들기 전에 살펴볼 함수를 좁힙니다.

Q3 뭐가 좋아지는 건가요? ④ 쌓인 데이터가 자산이 됩니다

쌓인 기록을 SQL로 꺼내 다음 판단과 분석에 씁니다.

화면은 DB 위의 보기 중 하나입니다. 같은 데이터를 읽기 전용 SQL과 CSV로 꺼내는 길을 처음부터 제품 기능으로 둡니다.

```
-- 최근 5영업일, 접수량 구간별 체결 p99: 초당 몇 건부터 밀리나
$ eum sql "SELECT in_tps, seconds, avg_p99_ms FROM ..."
 in_tps | seconds | avg_p99_ms
--------+---------+-----------
   1500 | 109,560 |        3.4
   3000 |   6,420 |        4.1
   4500 |   1,140 |        9.8
   6000 |     180 |      254.1

-- 평시와 밀린 구간의 함수별 CPU 비중(%)
-- 업무 지표와 함수 기록을 실행 번호로 조인
$ eum sql --csv "WITH w AS (...) SELECT ..." > slow.csv
 leaf          | normal_pct | slow_pct
---------------+------------+---------
 book_insert   |       15.2 |     63.4
 check_risk    |       34.0 |     14.0
 price_cross   |       21.8 |      9.6
 expire_orders |       11.1 |      8.2
```

설계 시안 · 두 질의는 같은 스키마의 합성 데이터로 실행해 확인 · 수치는 예시

이렇게 씁니다

활용	엮는 데이터	얻는 것
용량 판단	업무 지표 + 자원	몇 건부터 밀리는지 보고 증설 시점 판단
코드 개선	업무 지표 + 함수 기록	밀릴 때 비중이 커지는 함수 목록
배치 종료 예측	실행 이력 + 자원	정산이 끝나는 시각의 추세와 예측
이상 탐지	실행 이력	평소보다 오래 걸린 실행 자동 표시
배포 전후 비교	실행 이력 + 함수 기록	같은 작업의 함수 비중 변화로 성능 저하 발견

데이터의 주인은 우리입니다

스키마를 공개하고, 컬럼은 추가만 해서 기존 질의가 깨지지 않게 합니다. 보존 기간과 수집 주기도 설정으로 우리가 정합니다.

좋아지는 점

한 번 보고 끝나던 기록이 반복 비교와 분석의 재료가 됩니다.

Q3 뭐가 좋아지는 건가요? 확인 방법

효과는 숫자로 확인하고, 검증된 범위부터 넓힙니다.

확인하지 않은 절감 수치는 만들지 않습니다. 해커톤 검증판이 상용 제품 전체를 바로 대체한다고 가정하지도 않습니다.

좋아지는 것을 이렇게 확인합니다

좋아지는 것	확인 방법
① **운영 단순화**	도구 3개에서 보던 항목이 한 화면에 모두 나오는지 점검표로 확인
② **배치 운영**	병렬·합류·서버 넘기·재시도가 정의대로 돌고 그래프에 그대로 보이는지
③ **장애 조사**	지금 절차와 이음 절차로 같은 시나리오를 처리해 걸린 시간 비교. 지나간 구간의 함수 비중을 재현 없이 조회하는지
④ **데이터 활용**	운영 데이터 5종을 SQL로 조인하고 CSV로 내보내는지
수집 부담	함수 추적 자체의 CPU와 대상 서비스의 처리량·응답시간 변화 측정

연간 절감 시간 = 월 조사 대상 건수 [기입] × 건당 조사 시간 차이 × 12
연간 순절감액 = 실제 대체한 라이선스·유지보수 [기입] − 추가 운영·유지보수 비용 [기입]

초기 개발과 전환 비용은 따로 반영해 총비용과 회수 기간을 봅니다. 건수, 시간, 계약 금액은 제출 전 사내 확인이 필요한 값입니다.

사내 적용 순서

1단계
가상 환경 검증
해커톤 검증판으로 연결·과거 조회·함수 비교 확인

2단계
비운영 서버 시험
권한, 빌드 조건, 수집 부담, 장시간 안정성 점검

3단계
제한적 병행 운영
일부 작업에 기존 도구와 함께 써 보고 대체 범위 판단

운영 전에 갖출 것

로그인과 권한, 암호화 통신(TLS), 장애 복구와 이중화, 수집 누락과 보존 정책. 설계에 자리는 정해 두었고, 운영 투입 전에 구현하고 검증합니다.

비용 비교는 대체 범위만큼

병행 운영에서 실제로 대체한 기능에 한해 라이선스 절감액을 셉니다. 대신 담당자 유지보수 시간, 저장 공간, 보안 점검 비용을 함께 넣습니다.

원칙

작게 검증하고, 확인된 만큼 넓힙니다.

Q4 어떻게 가능한 건가요?

실행마다 번호를 붙이면, 모든 기록이 그 번호로 이어집니다.

새로 발명하는 기술은 없습니다. 검증된 부품을 쓰고, 우리가 만드는 것은 그 사이를 잇는 부분입니다.

번호가 이어지는 과정

1 **번호 발급**

스케줄러가 실행 1회마다 고유 번호를 만듭니다. 재시도는 새 번호로 구분합니다.

2 **실행 명령에 담아 전달**

서버의 에이전트가 명령과 번호를 받아 스크립트를 실행합니다.

3 **프로세스에 상속**

번호를 환경변수로 넘기면 스크립트가 띄운 C 서비스까지 이어집니다. 스크립트가 끝나고 서비스가 그룹을 벗어나도 번호는 남습니다.

4 **번호로 찾아 기록**

에이전트가 그 번호를 가진 프로세스를 찾아 자원과 함수를 기록하고, 서비스는 업무 지표에 번호를 붙여 보냅니다.

실행 결과	서비스 자원	업무 지표	함수 기록
시작·종료·재시도	CPU·메모리, 1초	접수·처리·대기·p99	10초 구간
01JBQ7…	01JBQ7…	01JBQ7…	01JBQ7…

어려운 일은 검증된 부품이 합니다

할 일	쓰는 부품	우리가 만드는 부분
함수 기록	리눅스 커널 내장 perf (초당 49회 표본)	대상 프로세스 선택, 결과를 접어 저장
저장·조회	SQLite (파일 하나짜리 DB)	공개 스키마, 분석용 뷰, 조회 API
실행 시각	검증된 cron 해석 라이브러리	선후행·재시도 판단, 실행 루프
차트	uPlot (경량 차트)	플로우 그래프, 플레임 그래프
배포	Go 단일 실행 파일	매니저·에이전트 역할 나누기

구성

batch-01 배치 서버 에이전트

match-01 체결 서버 에이전트

gw-01 게이트웨이 에이전트

→

매니저 스케줄러 · 수집 · 저장 · 화면

실행 상태 DB + 지표·함수 DB

연결은 에이전트 → 매니저 한 방향

Q4 어떻게 가능한 건가요? 하루, 2명

2명이 AI 세션 4개로, 하루에 검증판을 만듭니다.

대회 규칙상 코드는 당일 처음부터 씁니다. 사전에는 설계 문서, 작업카드, AI 지시문만 준비했습니다. 사람은 데이터 형식·API 합의와 검수를, AI 세션은 구현을 맡습니다.

당일 일정



범위

15:00까지 필수

정해진 시각 실행, 선후행·병렬·합류·재시도, 원격 실행, 서비스 추적, 자원·업무 지표 수집, 함수 기록, 타임머신·플레임 그래프, SQL·CSV, 에이전트 토큰

시간이 남으면

업무 지표 조건 실행(정산 대기), 함수 비교 표, 임계치 알림, 실행 강제 종료

이번에 안 함

로그인·권한, TLS, 이중화, 휴일 캘린더, 자바 함수 추적, 잠금·디스크 대기 분석

미리 확인한 것

함수 이름 복원	운영과 같은 조건(-O2, 프레임 포인터 없음)으로 빌드한 C 시험 프로그램에서 호출 경로를 정확히 복원
추적 부담	10초 구간당 CPU 약 0.06초, 코어 1개의 **약 0.6%**. 대상 프로그램 소요 시간은 측정 오차 안(13.3~13.4초 → 13.4초)
서비스 추적	스크립트 밖으로 빠져나간 프로세스를 환경변수 표식으로 찾음(일반 사용자 권한)
분석 질의	12쪽 SQL 2개를 같은 스키마의 합성 데이터로 실행
운영 서버	perf 사용 가능 확인

시험 프로그램 기준 값이며 실서비스 성능을 보장하지 않습니다. 본선에서 다시 측정하고 화면에 그대로 띄웁니다.

늦어지면 줄이는 순서

대시보드 장식 → 내보내기 → 구간 끌어 고르기(고정 구간으로) → 플레임 그래프(상위 함수표로). 실행 번호 연결과 함수 기록 수집은 줄이지 않습니다.

본선은 외부망과 상용 AI 도구로 개발하며, 사내 데이터와 운영 소스는 쓰지 않습니다. 시연은 컨테이너 3대와 합성 주문을 처리하는 시연용 C 서비스로 합니다. 서버 자원 값만 모의 값이고 실행·추적·함수 기록은 실제로 동작합니다.